

AI TRAINING – WEEK 2

DAY 2 ASSIGNMENT

Dictionaries, Nested Dictionaries, Counter & Comprehensions

Learn → Practice → Build → Explain

1. What You Will Learn

You will learn how dictionaries store data using key-value pairs. You will also practice nested dictionaries, Counter, and comprehensions.

- Dictionaries – keys, values, accessing and updating data
- Dictionary methods
- Nested dictionaries
- Counter for counting repeated values
- List and dictionary comprehensions
- Common errors and how to fix them
- Build student, contact, and word-frequency analyzers

2. Student Deliverable – Keep It Simple

Submit ONE Jupyter Notebook only.

File name: Week2_Day2_Dictionaries_YourName.ipynb

Your notebook should contain:

- Part A – Dictionary basics
- Part B – Dictionary methods
- Part C – Nested dictionaries
- Part D – Counter
- Part E – Comprehensions
- Part F – Debugging
- Part G – Student Analyzer
- Part H – Contact Analyzer
- Part I – Word Frequency Analyzer
- Part J – Explain what you learned

No separate Python files, Word/PDF files, or screenshots are required.

3. Part A – Dictionary Basics

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
```

- Print the complete dictionary.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
print(student)
```

- Print the student's name.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
print(student["name"])
```

- Print the student's marks.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
print(student["marks"])
```

- Change the marks to 90.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
student["marks"] = 90
print(student["marks"])
```

- Add a city key.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
student["city"] = "darsi"
print(student["city"])
```

- Remove the age key.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
del student["age"]
print(student)
```

- Print the final dictionary.

```
print(student)
#output
#{'name': 'Rahul', 'course': 'Python', 'marks': 85}
```

Think: Why is a dictionary useful for storing information about a student?

4. Part B – Dictionary Methods

Practice: keys(), values(), items(), get(), update(), pop()

- Print all keys.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
for keys in student.keys():
    print(keys)
```

- Print all values

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
for values in student.values():
    print(values)
```

- Print all key-value pairs using items().

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
for keys, values in student.items():
    print(keys, ":", values)
```

- Use get() to safely access a key.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
print(student.get("name"))
```

- Use update() to add or change information.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
student.update({"name": "devendra"})
print(student)
```

- Use pop() to remove one item.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
student.pop("name")
print(student)
```

5. Part C – Nested Dictionaries

```
students = {
    "student1": {"name": "Rahul", "marks": 85},
    "student2": {"name": "Priya", "marks": 92},
    "student3": {"name": "Arjun", "marks": 78}
}
```

- Print the complete dictionary

```
for keys, values in students.items():
    print(keys, values)
```

- Print Priya's name.

```
print(students["student2"]["name"])
```

- Print Priya's marks.

```
print(students["student2"]["marks"])
```

- Change Arjun's marks to 82.

```
students["student3"]["marks"]=82
print(students["student3"]["marks"])
```

- Add a city for each student.

```
students['student1'].update({"city":"darsi"})
students['student2'].update({"city":"vinukonda"})
students['student3'].update({"city":"bayyavaram"})
print(students["student1"]["city"])
print(students["student2"]["city"])
print(students["student3"]["city"])
```

- Print each student's name and marks using a loop.

```
for student in students.values():
    print(student["name"],":",student["marks"])
```

Challenge: Find the student with the highest marks.

```
highest = 0
top_student = ""
for student in students.values():
    if student["marks"] > highest:
        highest = student["marks"]
        top_student = student["name"]
print("Highest marks:", highest)
print("Student:", top_student)
```

6. Part D – Counter

Counter is available from the collections module and is useful for counting repeated values.

```
from collections import Counter
fruits = ["apple", "banana", "apple", "orange", "banana", "apple"]
```

- Create a Counter for fruits.

```
count=0
for i in fruits:
    count+=1
print(count)
or
```

```
count=Counter(fruits)
print(count)
output:
Counter({'apple': 3, 'banana': 2, 'orange': 1})
```

- Find the count of apple.

```
count=Counter(fruits)
print(count["apple"])
output:3
```

- Find the count of banana.

```
count=Counter(fruits)
print(count["banana"])
```

output:2

- Find the most common fruit.

```
count=Counter(fruits)
print(count.most_common(1))
output: [('apple', 3)]
```

- Try Counter with the string "programming".

```
from collections import Counter
string="programming"
count=Counter(string)
print(count)
```

7. Part E – Comprehensions

Comprehensions provide a short way to create collections.

Example:

```
numbers = [1, 2, 3, 4, 5]
```

```
squares = [n * n for n in numbers]
```

- Create a list of squares from 1 to 10.

```
lst=[i*i for i in range(1,11)]  
print(lst)
```

output:

[1, 4, 9, 16, 25, 36, 49, 64, 81,100]

- Create a list containing only even numbers from 1 to 20.

```
lst=[i for i in range(1,21)if i%2==0]  
print(lst)
```

output: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

- Create a dictionary where numbers 1–5 are keys and their squares are values.

```
dict={i:i*i for i in range(1,6)}  
print(dict)
```

output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

- Create a dictionary of student names and marks only for students who scored 80 or above.

```
students = {  
    "Rahul": 85,  
    "Priya": 92,  
    "Arjun": 78,  
    "Ananya": 88,  
    "Ravi": 65  
}  
result = {name: marks for name, marks in students.items() if marks >= 80}  
print(result)
```

output:

{'Rahul': 85, 'Priya': 92, 'Ananya': 88}

8. Part F – Quick Debugging

Find the error, fix it, run the code, and explain the mistake.

Debugging Problem 1

```
student = {"name": "Rahul", "age": 21}  
print(student["marks"])
```

ans:there is no key called "marks" in the dictionary.

Debugging Problem 2

```
student = {"name": "Rahul"}  
print(student[0])
```

ans:we cannot use indexing in the dictionary.

Debugging Problem 3

```
student = {"name": "Rahul"}
student.add("city", "Hyderabad")
```

ans: there is no add attribute in the dictionary .

```
students= {"name": "Rahul"}
students["city"]="hyderabad"
print(students)
```

Debugging Problem 4

```
numbers = [1, 2, 3, 4]
squares = {n: n * n for n in numbers}
print(squares)
```

ans:at the ending of squares used "]" instead of "}".

```
numbers = [1, 2, 3, 4]
squares = {n: n * n for n in numbers}
print(squares)
```

9. Part G – Hands-on Activity: Student Analyzer

Start with: student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}

Your program should:

- Display student details.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
for keys,values in student.items():
    print(keys,":",values)
```

- Check whether the student passed or failed.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
if student["marks"]>50:
    print("pass")
else:
    print("fail")
```

- Assign a grade based on marks.

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
if student["marks"]>=80 and student["marks"]<=100:
    student["grade"]="A"
print(student)
```

- Allow marks to be updated.
- Display the final student information.

```
for keys, values in student.items():
    print(f"{keys}:{values}")
```

output: name:Rahul
age:21
course:Python
marks:85
grade:A

Bonus: Store 3–5 students in a nested dictionary and find the highest scorer.

```
students = {
    "student1": {"name": "deva", "marks": 85},
    "student2": {"name": "devendra", "marks": 92},
    "student3": {"name": "dini", "marks": 78},
    "student4": {"name": "dinesh", "marks": 88},
    "student5": {"name": "Ravi", "marks": 65}
}
highest = 0
highest_name = ""
for student in students.values():
    if student["marks"] > highest:
        highest = student["marks"]
        highest_name = student["name"]
print("Highest scorer:", highest_name)
print("Marks:", highest)
```

output:

Highest scorer: devendra

Marks: 92

10. Part H – Hands-on Activity: Contact Analyzer

```
contacts = {"Rahul": "9876543210", "Priya": "9123456780", "Arjun": "9988776655"}
```

- Add a new contact

```
contacts = {"Rahul": "9876543210", "Priya": "9123456780", "Arjun": "9988776655"}
contacts["devendra"] = "9391766956"
```

- Search for a contact by name.

```
contacts["devendra"] = "9391766956"
name = input("Enter name: ")
for key, value in contacts.items():
    if name.lower() == key.lower():
        print("Phone number:", value)
```

- Update a phone number.

```
contacts["devendra"] = '9876543211'
```

- Delete a contact.

```
del contacts["Priya"]
print(contacts)
```

- Display all contacts.

```
for keys, values in contacts.items():
    print(f"{keys}:{values}")
```

output: Rahul:9876543210
Arjun:9988776655
devendra:9876543211

Bonus: Store phone, email, and city for each contact using a nested dictionary.

```
contacts = {"Rahul": "9876543210", "Priya": "9123456780", "Arjun": "9988776655"}
contacts = {
    "Rahul": {
        "phone": "9876543210",
        "email": "rahul@gmail.com",
        "city": "Hyderabad"
    },
    "Priya": {
        "phone": "9123456780",
        "email": "priya@gmail.com",
        "city": "Bangalore"
    },
    "Arjun": {
        "phone": "9988776655",
        "email": "arjun@gmail.com",
        "city": "Chennai"
    }
}
for keys, values in contacts.items():
    print(f"{keys}:{values}")
```

output:

Rahul:{'phone': '9876543210', 'email': 'rahul@gmail.com', 'city': 'Hyderabad'}

Priya:{'phone': '9123456780', 'email': 'priya@gmail.com', 'city': 'Bangalore'}

Arjun:{'phone': '9988776655', 'email': 'arjun@gmail.com', 'city': 'Chennai'}

11. Part I – Hands-on Activity: Word Frequency Analyzer

Example sentence: "python is easy and python is powerful"

- Convert the sentence into words.

```
sentence="python is easy and python is powerful"
words=sentence.split()
print(words)
output: ['python', 'is', 'easy', 'and', 'python', 'is', 'powerful']
```

- Use Counter to count each word.
- Display the word counts.

```
from collections import Counter
sentence="python is easy and python is powerful"
words=sentence.split()
count = Counter(words)
print("Word counts:")
for word, frequency in count.items():
    print(word, ":", frequency)
```

output;

Word counts:

```
python : 2
is : 2
easy : 1
and : 1
powerful : 1
```

- Find the most frequently used word.

```
from collections import Counter
sentence="python is easy and python is powerful"
words=sentence.split()
count=Counter(words)
print(count.most_common(1))
```

output:

```
[('python', 2)]
```

- Bonus: convert all words to lowercase before counting

```
from collections import Counter
sentence="PYthon is easy and python is powerful"
words=sentence.lower().split()
count=Counter(words)
print(count)
```

12. Part J – Explain in Your Own Words

1. What is a dictionary?

Answer: dictionary is a collection which is used to store key values pairs.

2. What is a key-value pair?

Answer:

3. Why are dictionaries useful?

Answer: dictionaries are usefull to store data in key value pairs.we access the values by using keys.

4. What is a nested dictionary?

Answer: a dictionary inside another dictionary.

5. Why is Counter useful?

Answer: counter is useful count the number of items in dictionary.

6. What is a comprehension?

Answer: compressing the code into single line.

7. What is the difference between a normal loop and a comprehension?

Answer: In normal loop we need to write the code in more than three line for comprehension in single line is enough for same code.

8. When would you use a dictionary instead of a list?

Answer: when I need to store the values using keys like student details.

13. Student Submission Checklist

- I practiced dictionary creation and access.
- I used dictionary methods.
- I practiced nested dictionaries.
- I used Counter.
- I practiced comprehensions.
- I fixed the debugging problems.
- I completed the Student Analyzer.
- I completed the Contact Analyzer.

- I completed the Word Frequency Analyzer.
- I explained the concepts in my own words.
- My notebook runs without major errors.

14. After This Assignment, You Should Be Able To

- Store structured information using dictionaries.
- Access and update dictionary values.
- Work with nested dictionaries.
- Count repeated values using Counter.
- Create lists and dictionaries using comprehensions.
- Choose dictionaries when data is naturally represented as key-value pairs.
- Build small real-world data analyzers.

Key Principle: Don't just learn dictionary syntax. Learn how to organize, search, update, and analyze data.